



UNIVERSITÉ D'ÉVRY PARIS-SACLAY

MASTER 1 COMPUTER & NETWORK SYSTEMS

TRAVAIL D'ÉTUDE ET DE RECHERCHE

Optimisation du TSP

Recherche locale et parallélisation

Aravindan DETCHANAMOURTTY

20220383@etud.univ-evry.fr

Arvinth MAHESANANTHAN

20212814@etud.univ-evry.fr

Shayan VADOUD

20223926@etud.univ-evry.fr

Encadrant : Eric Angel
Année universitaire 2025-2026

Glossaire

1. **Cycle hamiltonien :**
Chemin dans un graphe passant par chaque sommet (chaque ville) exactement une fois et revenant au sommet de départ. Le TSP consiste à trouver le cycle hamiltonien de longueur minimale.
2. **NP-difficile (NP-dur) :**
Problème de calcul dont la résolution exacte demande un temps qui croît de façon exponentielle avec la taille des données (comme le TSP).
3. **Heuristique :**
Méthode de calcul rapide pour obtenir une solution satisfaisante à un problème complexe, sans garantie d’optimalité.
4. **Métaheuristique :**
Algorithme de haut niveau qui pilote et coordonne plusieurs heuristiques pour explorer l’espace des solutions.
5. **Solveur :**
Logiciel regroupant les algorithmes nécessaires pour résoudre un problème donné et produire une solution exploitable (ici, un tour pour le TSP).
6. **Nearest Neighbor (NN) :**
Algorithme de construction glouton créant un trajet en choisissant toujours la ville non visitée la plus proche.
7. **2-opt / 3-opt / k -opt :**
Heuristiques d’amélioration par modification locale du chemin (échange de 2, 3 ou k arêtes) pour éliminer les croisements.
8. **k -NN (k -Nearest Neighbors) :**
Filtrage limitant les modifications aux k voisins les plus proches de chaque ville pour gagner en rapidité.
9. **Graphe de Conflits :**
Modèle où les sommets sont des mouvements k -opt et les arêtes représentent des incompatibilités empêchant leur application simultanée.
10. **MIS (Maximum Independent Set) :**
Dans un graphe $G = (V, E)$, sous-ensemble $S \subseteq V$ de cardinalité maximale tel qu’aucune arête ne relie deux sommets de S . C’est un problème NP-difficile en général ; dans ce TER, il s’applique au graphe de conflits entre mouvements 2-opt compatibles.
11. **Parallélisme algorithmique :**
Capacité de l’algorithme à identifier et à regrouper plusieurs actions logiques indépendantes pour les appliquer en un seul pas d’itérations.
12. **Parallélisme matériel :**
Utilisation réelle de plusieurs cœurs de processeur (multiprocessing) pour effectuer des calculs simultanément et réduire le temps d’exécution.
13. **GIL (Global Interpreter Lock) :**
Verrou n’autorisant qu’un seul thread à exécuter du bytecode Python à la fois. Les threads ne parallélisent pas efficacement le calcul CPU intensif ; les processus (multiprocessing) contournent cette limite.

14. **Speedup :**
Rapport entre le temps d'exécution en séquentiel et le temps en parallèle, mesurant l'efficacité de l'optimisation.
15. **VNS (Variable Neighborhood Search) :**
Métaheuristique hybride qui alterne entre recherche locale (2-opt / 3-opt) et perturbation pour éviter les minima locaux.
16. **Gap (%) :**
Écart relatif entre le coût d'une solution trouvée et la meilleure solution connue.
17. **Minimum local :**
Solution dont le voisinage exploré (ex. 2-opt, 3-opt) ne permet plus d'amélioration (pas nécessairement optimale globalement).
18. **Double-bridge (kick) :**
Mouvement 4-opt ciblé utilisé comme perturbation aléatoire pour sortir d'un minimum local et relancer la descente locale.
19. **Frontière de Pareto :**
Ensemble de solutions non dominées dans un compromis multi-critères : on ne peut pas améliorer l'un sans dégrader l'autre.
20. **TSPLIB :**
Bibliothèque de jeux de test standard pour le TSP, avec instances et valeurs de référence.

Table des matières

1	Introduction et objectif	4
1.1	Contexte	4
1.2	Objectif du travail d'étude et de recherche	4
1.3	Annonce du plan	4
2	État de l'art et problématique	5
2.1	Théorie du TSP	5
2.2	Recherche locale	5
2.3	Problématique	6
3	Méthodologie et baseline	6
3.1	Format des données	6
3.2	Solution initiale (Nearest Neighbor)	6
3.3	2-opt séquentiel	7
4	Optimisation par lots et réduction de voisinage	8
4.1	k -Nearest Neighbors (k -NN)	8
4.2	Graphe de conflits	9
4.3	Sélection par MIS (Maximum Independent Set)	9
4.4	Application en lots (batch)	10
5	Recherche locale avancée : 3-opt parallèle	10
5.1	Complexité et les 7 cas de reconnexion	10
5.2	Architecture multi-processus (mémoire partagée)	11
5.3	Modèle Parallel-Step et synchronisation	11
6	Métaheuristique VNS et double-bridge	12
6.1	Limites et minima locaux	12
6.2	Double-bridge (4-opt) et perturbations	12
6.3	Variable Neighborhood Search (VNS) hybride	13
7	Évaluation expérimentale et analyse	13
7.1	Protocole et matériel	13
7.2	Analyse du speedup	14
7.3	Analyse de qualité (Gap %)	15
7.4	Analyse du temps	17
7.5	Analyse de Pareto et stabilité	17
8	Conclusions et perspectives	19
8.1	Résumé des contributions	19
8.2	Auto-évaluation et prise de recul	19
8.3	Perspectives : GPU, métaheuristiques et apprentissage	20

1 Introduction et objectif

1.1 Contexte

Le problème du voyageur de commerce (Traveling Salesman Problem - TSP) est un problème d'optimisation combinatoire classé NP-difficile⁽²⁾. L'objectif est de déterminer le cycle hamiltonien⁽¹⁾ de longueur minimale reliant un ensemble de n villes, en visitant chaque sommet une seule fois avant de retourner au point de départ.

Pour résoudre ce problème, on utilise classiquement des heuristiques⁽³⁾ de recherche locale, telles que les algorithmes 2-opt ou 3-opt⁽⁷⁾. Bien que ces méthodes fournissent des solutions de bonne qualité, elles se heurtent à une limite majeure : l'explosion du coût calculatoire liée à l'exploration exhaustive du voisinage. Lorsque la taille des instances augmente, le traitement séquentiel devient trop lent pour obtenir des résultats en un temps raisonnable, ce qui motive des approches de réduction de voisinage et de parallélisation.

Ce travail s'inspire directement de recherches académiques majeures, notamment les travaux de Verhoeven et al. [1][2] et l'étude récente de Qiao et al. [3]. Ces références ont guidé nos choix techniques, particulièrement concernant la réduction de voisinage (k -NN⁽⁸⁾) et la parallélisation des mouvements d'amélioration.

1.2 Objectif du travail d'étude et de recherche

L'objectif principal de cette étude est de concevoir un solveur⁽⁵⁾ capable de dépasser les limites des méthodes classiques pour traiter efficacement de grandes instances du TSP. Notre approche repose sur deux axes majeurs :

- **Le parallélisme** : Plutôt qu'une exploration séquentielle, notre solveur utilise un graphe de conflits⁽⁹⁾ pour identifier des ensembles de mouvements disjoints (via un MIS⁽¹⁰⁾). Cette structure permet d'appliquer simultanément de multiples modifications en une seule itération, réduisant drastiquement le temps de calcul.
- **L'hybridation** : Afin d'éviter les minima locaux, nous avons développé une architecture combinant la rapidité du 2-opt + k -NN, la puissance du 3-opt parallèle et la robustesse d'un algorithme hybride inspiré du VNS⁽¹⁵⁾.

Le but final est de démontrer qu'en combinant réduction de voisinage, exécution parallèle et métaheuristique⁽⁴⁾, il est possible d'obtenir un compromis temps/qualité supérieur aux méthodes classiques sur des instances de grande taille.

1.3 Annonce du plan

Ce rapport est structuré comme suit : après une présentation de l'état de l'art et de la problématique théorique (Partie 2), nous détaillerons notre méthodologie et les performances de notre base de référence (Partie 3). Les parties 4, 5 et 6 constitueront le cœur technique de notre travail, en présentant successivement l'optimisation par lot, le parallélisme du 3-opt et l'hybridation via le VNS. Enfin, nous analyserons l'ensemble des résultats expérimentaux (Partie 7) avant de conclure sur les perspectives de ce projet (Partie 8).

2 État de l'art et problématique

2.1 Théorie du TSP

D'un point de vue formel, le problème du voyageur de commerce est modélisé par un graphe complet pondéré $G = (V, E)$, où V représente l'ensemble des n villes et E l'ensemble des arêtes qui les relient. À chaque arête est associé un coût correspondant à la distance euclidienne entre deux sommets.

L'objectif est de trouver une permutation des sommets formant un cycle hamiltonien de coût minimal. La difficulté majeure réside dans l'explosion combinatoire de l'espace de recherche : pour n villes, le nombre de cycles distincts possibles est de $(n - 1)!/2$. Cette croissance factorielle rend la résolution exacte impraticable par force brute dès que n dépasse une vingtaine de villes.

2.2 Recherche locale

La recherche locale repose sur le principe des heuristiques k -opt. À partir d'un trajet initial, l'algorithme tente d'améliorer la solution de manière itérative en modifiant k arêtes du cycle.

- **Le mouvement 2-opt** : On supprime deux arêtes non adjacentes et on reconnecte les segments de la seule manière possible pour reformer un cycle. Mathématiquement, cela revient à inverser l'ordre des villes dans le segment compris entre les deux cassures. C'est la méthode la plus rapide pour éliminer les croisements de routes.

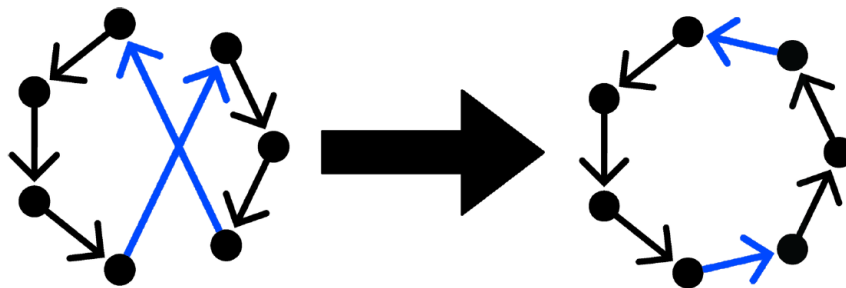


FIGURE 1 – Illustration du mouvement 2-opt.

- **Le mouvement 3-opt** : On supprime trois arêtes simultanément, ce qui offre sept schémas de reconnexion possibles. Bien que plus complexe et coûteux en temps de calcul, le 3-opt permet d'explorer un voisinage plus profond et d'obtenir des solutions de meilleure qualité que le 2-opt.
- **Le mouvement k -opt** : On généralise les principes précédents en retirant simultanément k arêtes du cycle, ce qui découpe le tour en k segments. Pour un k -opt donné, il existe alors au plus $2^{k-1}(k - 1)!$ schémas de reconnexion possibles (dont l'un restitue le tour initial).

2.3 Problématique

Le défi majeur de ce projet réside dans le passage à l'échelle. Bien que le 2-opt soit performant, son exécution séquentielle constitue une contrainte majeure : on ne modifie qu'une seule paire d'arêtes à la fois, sans exploiter les capacités multi-cœurs des processeurs modernes.

De plus, l'exploration complète du voisinage devient trop coûteuse sur de grandes instances. La problématique centrale de ce TER est donc la suivante :

Comment réduire intelligemment l'espace de recherche tout en orchestrant une exécution parallèle conséquente des changements qui permettent de raccourcir le trajet ?

Pour y répondre, nous distinguons le **parallélisme algorithmique**⁽¹¹⁾ (combinaison de mouvements compatibles en un seul pas) du **parallélisme matériel**⁽¹²⁾ (répartition du calcul sur plusieurs cœurs).

3 Méthodologie et baseline

3.1 Format des données

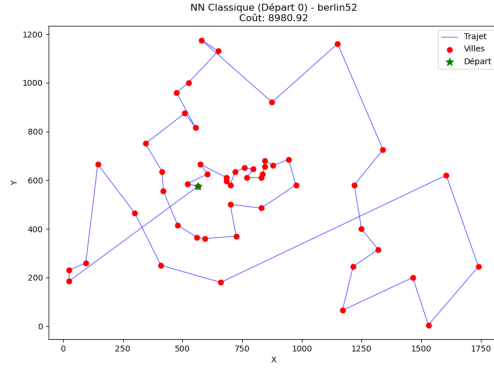
Pour évaluer notre solveur, nous utilisons la bibliothèque de référence **TSPLIB**⁽²⁰⁾. Les instances (comme **Berlin52**, **Ch150** ou **Pr1002**) fournissent les coordonnées x et y de chaque ville. Notre programme calcule ensuite la distance euclidienne entre chaque paire de villes pour construire la matrice des distances nécessaire aux calculs.

L'utilisation de ces instances standards nous permet de comparer nos résultats avec les solutions optimales connues et de mesurer précisément l'écart (gap⁽¹⁶⁾).

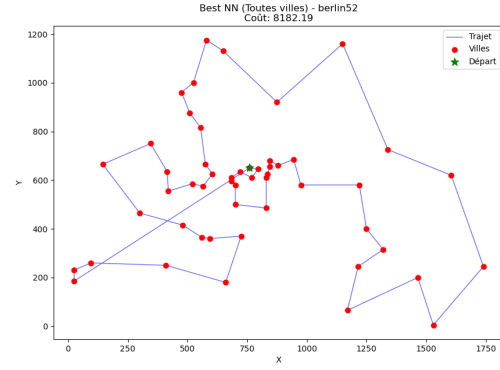
3.2 Solution initiale (Nearest Neighbor)

La première étape consiste à générer un trajet initial rapidement. Nous utilisons l'heuristique du **Plus Proche Voisin** (Nearest Neighbor⁽⁶⁾). C'est un algorithme glouton qui part d'une ville arbitraire et choisit systématiquement la ville non visitée la plus proche jusqu'à revenir au point de départ.

Bien que très rapide, cette méthode dépend fortement de la ville de départ. Pour optimiser cette base, nous avons implémenté une variante "**Best-NN**" qui teste chaque ville comme point de départ possible et ne conserve que le meilleur tour trouvé.



(a) Départ ville 0 (NN)



(b) Meilleur départ (Best-NN)

FIGURE 2 – Comparaison entre le Nearest Neighbor classique et son optimisation (Berlin52).

3.3 2-opt séquentiel

Une fois le trajet initial généré, nous appliquons une recherche locale 2-opt séquentielle en appliquant la logique du *first improvement*. Chaque mouvement est défini par deux indices i et j sur le tour : on supprime les arêtes reliant v_{i-1} à v_i et v_j à v_{j+1} , on inverse le segment de villes entre i et j , puis on reconnecte le cycle. À chaque étape, on parcourt les couples (i, j) jusqu'au premier mouvement améliorant, on l'applique, puis on recommence sur le tour mis à jour, jusqu'à optimum local. (Code annexe : A, B).

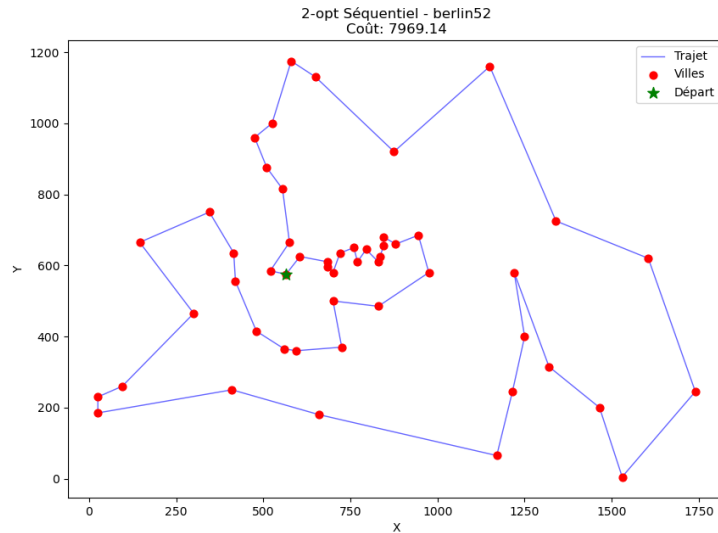


FIGURE 3 – Trajet optimisé par la recherche locale 2-opt classique (Berlin52).

Le tableau suivant présente les performances de notre baseline sur l'instance Berlin52 (52 villes) :

Algorithme	Distance	Ecart (Gap %)	Temps (ms)
Nearest Neighbor (Départ 0)	8981	+19,08%	5,04 ms
Best Nearest Neighbor (Tous départs)	8182	+8,49%	181,49 ms
2-opt Séquentiel (Baseline)	7670	+1,69%	96,71 ms

TABLE 1 – Performance des algorithmes de base sur l'instance Berlin52 (Optimum : 7542)

L'efficacité de la recherche locale est illustrée par le passage de 181,49ms pour le Best-NN à 96,71ms supplémentaires pour le 2-opt sur cette petite instance. Cependant, cette dynamique temporelle suggère que sur des instances plus larges (de l'ordre de 1000 villes), ces délais passeraient de quelques millisecondes à plusieurs secondes. Ce constat justifie pleinement le passage aux stratégies de parallélisation pour maintenir des performances acceptables à grande échelle.

4 Optimisation par lots et réduction de voisinage

4.1 k -Nearest Neighbors (k -NN)

Le passage à de grandes instances (plusieurs milliers de villes) rend l'exploration exhaustive du voisinage en $O(n^2)$ impraticable.

Pour y remédier, nous limitons la recherche locale aux k plus proches voisins de chaque ville. Cette stratégie de réduction d'espace de recherche [3] permet de passer à une complexité de $O(k \cdot n)$, où k est une constante. (Code annexe : E)

Le choix de k est crucial : il doit être assez grand pour ne pas rater des cas d'amélioration, mais assez petit pour garantir la rapidité. Comme le montre la figure 4, nous avons identifié un **point d'inflexion** dans la courbe du compromis temps/qualité aux alentours de $k = 15$ pour l'instance Berlin52.

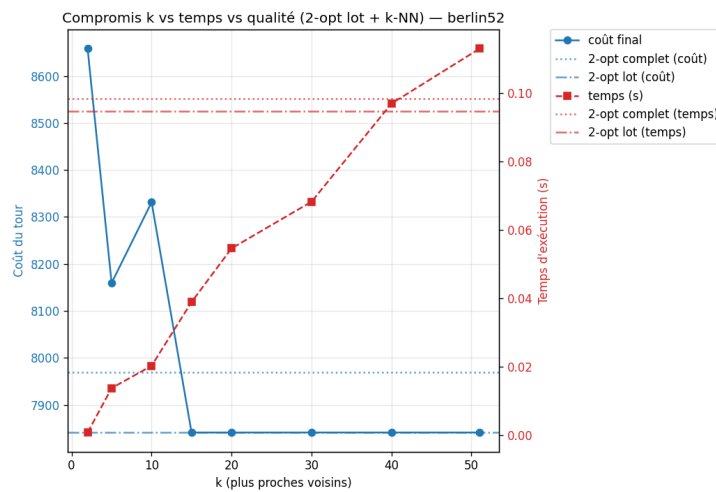


FIGURE 4 – Analyse du compromis entre temps de calcul et qualité de solution en fonction de k .

4.2 Graphe de conflits

Une fois le voisinage réduit par k -NN, l'objectif est d'appliquer un maximum de mouvements d'amélioration simultanément. Cependant, certains mouvements 2-opt peuvent être valides indépendamment mais ne pas être compatibles entre eux si les segments à inverser se chevauchent ou ne sont pas séparés par une arête intacte du tour. Leur application concurrente pourrait alors ne plus correspondre à une composition valide de 2-opt disjoints, et la somme des gains théoriques ne refléterait plus le gain réel obtenu.

Formellement, on dit que deux mouvements 2-opt définis par les segments $[i_1, j_1]$ et $[i_2, j_2]$ (avec $i_1 < i_2$) sont compatibles si et seulement si $j_1 + 1 < i_2$, soit lorsque les deux segments sont disjoints et séparés par au moins une arête intacte du tour. (Code annexe : C)

Pour orchestrer cela, on construit un **graphe de conflits** [1] où :

- Chaque sommet du graphe représente un mouvement 2-opt améliorant.
- Chaque arête entre deux sommets représente un conflit entre ces deux mouvements.

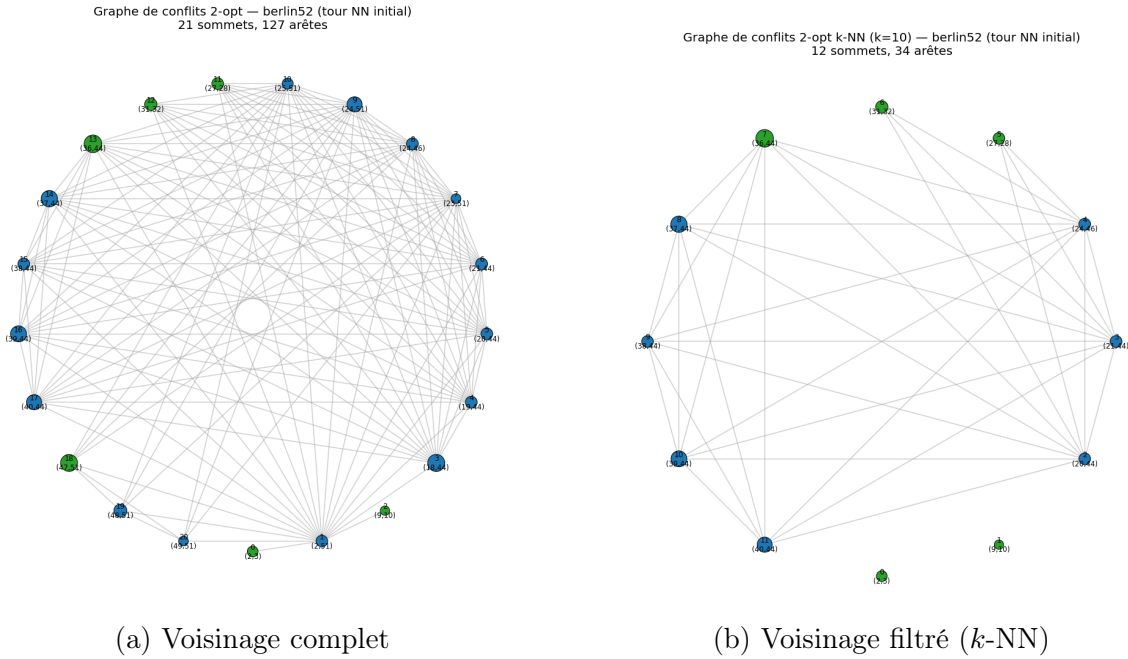


FIGURE 5 – Impact de la réduction de voisinage sur la structure du graphe de conflits.

4.3 Sélection par MIS (Maximum Independent Set)

Pour extraire un ensemble de mouvements applicables simultanément, on recherche un **Ensemble Indépendant Maximal (MIS)** dans le graphe de (Code annexe : D). Un ensemble indépendant est ici un groupe de mouvements qui ne sont pas reliés par une arête, garantissant ainsi qu'ils peuvent être appliqués sans interférences.

La recherche d'un MIS exact étant coûteuse, on utilise une heuristique gloutonne : les candidats sont triés par gain décroissant, puis on retient successivement le premier non exclu, et on écarte ce mouvement et tous ses voisins du graphe de conflits. On répète ainsi jusqu'à épuisement de la liste.

Cette approche permet d’extraire rapidement un lot de mouvements indépendants, en privilégiant les meilleurs gains à chaque itération.

4.4 Application en lots (batch)

L’étape finale de cette phase est l’application en lot. Plutôt que de scanner à nouveau tout le tour après chaque modification (comme en séquentiel), on applique l’ensemble des mouvements du MIS en une seule étape, ce qui permet d’optimiser le tour en un minimum d’itérations. (Code annexe : D)

C’est ici qu’on obtient notre première accélération significative par rapport à la base-line, en réduisant drastiquement le nombre de recalculs nécessaires entre deux phases de recherche.

Algorithme	Distance	Écart (Gap %)	Temps (ms)
2-opt Séquentiel (Baseline)	7670	+1,69%	96,71 ms
2-opt Lot + K-NN	7841	+3,97%	42,81 ms

TABLE 2 – Comparaison entre la recherche locale classique et l’optimisation par lot avec filtrage K-NN (Berlin52).

Le tableau 2 met en évidence l’efficacité du filtrage k -NN couplé à l’application par lot. Sur l’instance Berlin52, le temps d’exécution est divisé par plus de deux. Bien que l’élagage du voisinage induise une légère augmentation du Gap (passant de +1,69% à +3,97%), ce compromis est avantageux car il permet de traiter des instances beaucoup plus larges tout en conservant une qualité de solution acceptable.

5 Recherche locale avancée : 3-opt parallèle

5.1 Complexité et les 7 cas de reconnexion

L’heuristique 3-opt constitue une extension naturelle mais nettement plus puissante du 2-opt. Là où le 2-opt se contente d’inverser des segments pour éliminer les croisements simples, le 3-opt retire simultanément trois arêtes du cycle. (Code annexe : F)

D’un point de vue théorique, cette puissance se traduit par :

- **Une complexité accrue** : L’exploration exhaustive du voisinage passe de $O(n^2)$ à $O(n^3)$, rendant le traitement séquentiel lent sur des instances dépassant quelques centaines de villes.
- **Profondeur de voisinage** : Une fois trois arêtes supprimées, il existe alors 7 manières distinctes de reconnecter ces segments pour former un nouveau cycle hamiltonien. Contrairement au 2-opt (qui n’a qu’une seule autre reconnexion possible), le 3-opt explore donc un voisinage beaucoup plus vaste.

L’utilisation du 3-opt permet d’échapper à de nombreux minima locaux où le 2-opt reste bloqué, mais justifie un parallélisme matériel pour rester compétitif en temps de calcul [2].

5.2 Architecture multi-processus (mémoire partagée)

Pour réduire le coût du voisinage 3-opt (restreint par k -NN), nous avons implémenté une architecture parallèle basée sur le module `multiprocessing` (`ProcessPoolExecutor`). Contrairement aux *threads* Python, limités par le GIL⁽¹³⁾ pour le calcul CPU intensif, les *processus* permettent d'exploiter plusieurs cœurs de manière simultanée.

Le modèle s'appuie sur une mémoire partagée pour le trajet courant et une initialisation unique des données lourdes :

- **Mémoire partagée** : le tour est stocké dans un `multiprocessing.Array` ; tous les workers le lisent à chaque itération, tandis que le processus parent le met à jour après chaque mouvement appliqué.
- **Données répliquées une fois** : l'instance TSP et les listes k -NN sont chargées dans chaque worker via `init_worker`, afin d'éviter de les recopier à chaque tâche.
- **Chunks sur i** : les premiers indices de rupture i sont répartis en tranches ; pour chaque i , les indices j et k ($i < j < k$) sont explorées via le voisinage k -NN. Le parent retient le meilleur gain et applique un unique mouvement 3-opt (*best improvement*).

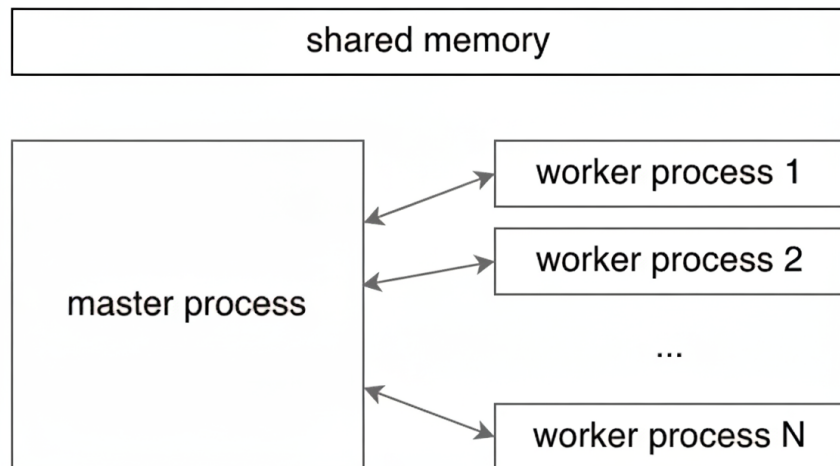


FIGURE 6 – Architecture parent - workers avec mémoire partagée.

5.3 Modèle Parallel-Step et synchronisation

Pour orchestrer le travail des différents processus, nous nous inspirons du modèle **Parallel-Step** [2] : à chaque itération, les workers évaluent le voisinage en parallèle, puis le processus parent effectue un unique pas de recherche locale. (Code annexe : G)

1. **Exploration parallèle** : les processus explorent simultanément des tranches d'indices i ; pour chaque i , ils testent des mouvements (i, j, k) admissibles dans le voisinage k -NN et retiennent le meilleur gain local (*best improvement* sur la tranche).
2. **Synchronisation** : le parent attend la fin de toutes les tâches (`executor.map`) avant d'agréger les résultats.

3. **Mise à jour unique** : seul le mouvement 3-opt de gain maximal est appliqué ; le tour mis à jour est recopié dans la mémoire partagée pour l'itération suivante.

Nous verrons dans la partie suivante comment intégrer ce modèle dans une boucle métaheuristique plus large (VNS) pour dépasser certains minima locaux.

6 Métaheuristique VNS et double-bridge

6.1 Limites et minima locaux

Malgré la puissance du 3-opt parallèle, la recherche locale souffre d'un défaut structurel : elle est gloutonne. Dès qu'elle atteint un état où aucun mouvement dans son voisinage immédiat ne permet de réduire la distance, elle s'arrête.

On appelle cet état un **minimum local**⁽¹⁷⁾. Sur des instances complexes, il existe des milliers de minima locaux dont la qualité est très éloignée de l'optimum global. Pour dépasser ces barrières, augmenter la vitesse d'exploration (parallélisme matériel) ne suffit plus ; il faut adopter une stratégie capable de s'extraire de ces optima locaux pour explorer de nouvelles zones de l'espace de recherche.

6.2 Double-bridge (4-opt) et perturbations

Un 4-opt exhaustif implique de nombreux schémas de reconnexion [3], et on ne retient donc ici qu'un motif fixe appelé **double-bridge**⁽¹⁸⁾ pour limiter le coût : on retire quatre arêtes et on reconnecte les segments selon l'ordre *A-D-C-B-E* (Code annexe : H). Contrairement au 2-opt et au 3-opt en descente, nous l'utilisons comme **perturbation** (*kick*), sans chercher un gain immédiat.

- **Sortie des minima locaux** : ce mouvement modifie profondément le tour ; il n'est en pratique pas annulé par un seul 2-opt ou 3-opt, ce qui force l'exploration d'une autre configuration.
- **Diversification modérée** : contrairement à un redémarrage aléatoire complet, une partie des bonnes sous-séquences est conservée tout en ouvrant de nouvelles zones du voisinage.

Lorsqu'une descente locale (2-opt / 3-opt) ne parvient plus à améliorer le meilleur tour connu, ce mouvement sert de *kick* : on applique un double-bridge à coupures aléatoires, puis on relance une phase d'optimisation locale sur le nouveau tour. (Code annexe : H).

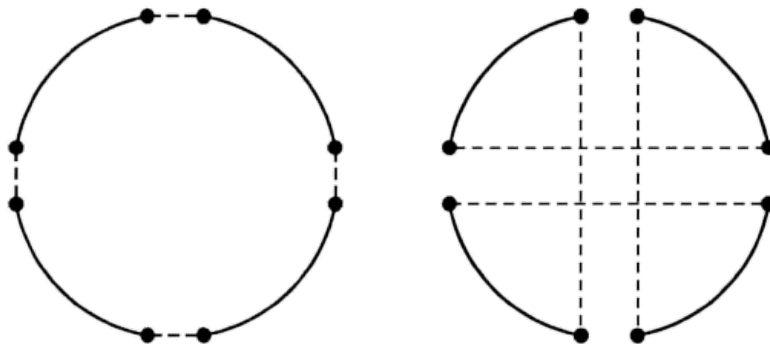


FIGURE 7 – Mécanisme du mouvement double-bridge (4-opt) : suppression de 4 arêtes et reconnexion croisée.

6.3 Variable Neighborhood Search (VNS) hybride

L’algorithme final regroupe les modules développés dans une boucle inspirée de la *Variable Neighborhood Search* : alternance entre **intensification** (descente locale) et **diversification** (*kick*).

1. **Descente hiérarchique** : le 2-opt par lot (k -NN) nettoie rapidement le tour ; le 3-opt parallèle (k -NN) l’affine jusqu’à un minimum local du voisinage exploré.
2. **Diversification** : si cette descente n’améliore plus le meilleur tour connu, un double-bridge aléatoire est appliqué à ce meilleur tour pour changer de zone de recherche.
3. **Critère d’arrêt et d’acceptation** : toute amélioration met à jour la référence et réinitialise le compteur de kicks ; sinon on incrémente ce compteur jusqu’à une limite `max_kicks`, puis on relance la descente.

Cette hybridation compense la nature gloutonne de la descente par une perturbation ciblée, au prix d’un compromis entre précision locale et exploration globale.

7 Évaluation expérimentale et analyse

7.1 Protocole et matériel

Afin de quantifier les gains apportés par nos différentes optimisations, nous avons mis en place un protocole de tests basé sur les instances de référence de la bibliothèque **TS-PLIB**, qui fournit des jeux de données standardisés et des valeurs optimales de référence pour comparer objectivement la qualité des solutions.

- **Configuration matérielle** : Les tests ont été effectués sur une machine équipée d’un processeur Intel Core i5-1135G7 (2,40GHz) et de 8Go de mémoire vive. L’exploitation du parallélisme matériel a été testée sur 4 cœurs physiques (8 threads).

- **Environnement logiciel** : L'implémentation a été réalisée en Python 3.10, utilisant principalement les bibliothèques NumPy pour les calculs matriciels et `multiprocessing` pour le parallélisme matériel.
- **Instances de test** : Nous avons sélectionné un panel d'instances de complexité croissante : `berlin52` (petite), `pcb442` (moyenne) et `pr1002` (grande), afin d'observer le comportement du solveur face à l'augmentation de la dimension du problème.

Chaque benchmark est répété 5 fois afin d'obtenir des temps d'exécution moyens stables et de minimiser les variations liées au système d'exploitation.

7.2 Analyse du speedup

Le **speedup**⁽¹⁴⁾ $S = T_s/T_p$ compare le temps du 3-opt parallèle sur 1 processus (T_s) à celui sur p processus (T_p).

- **Petites instances** (`berlin52`) : S atteint environ 2 sur 4–8 processus, mais reste nettement sous-linéaire : le surcoût (création de processus, synchronisation, mémoire partagée) limite le gain lorsque le volume de calcul par itération est faible.
- **Grandes instances** (`pr1002`, `pcb442`) : $S \approx 2,5$ sur 4 processus ; au-delà, les gains se tassent voire diminuent (8 processus), ce qui traduit une scalabilité limitée par l'overhead, l'équilibrage des chunks et le plafond matériel.

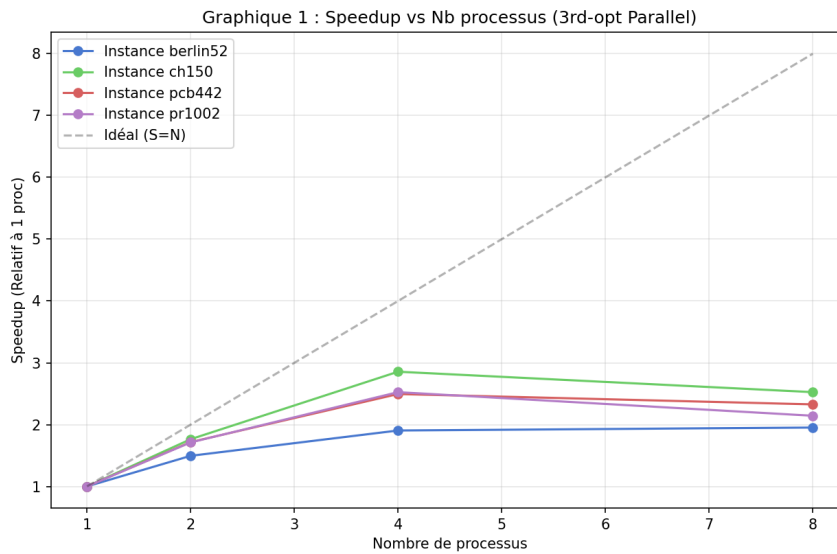


FIGURE 8 – Évolution du speedup en fonction du nombre de processus pour différentes instances (3-opt parallèle).

Toutefois, le speedup reste limité par une part séquentielle du cycle : synchronisation des workers, réduction du meilleur gain et mise à jour du tour partagé après chaque itération. Sur la figure 8, les gains se tassent après 4 processus, ce qui est cohérent avec le nombre de cœurs physiques de la machine de test. Au-delà, on exploite surtout l'*hyperthreading* (threads logiques supplémentaires), dont le gain est limité pour du calcul CPU pur, ce qui explique la stagnation du speedup.

7.3 Analyse de qualité (Gap %)

L'efficacité qualitative de nos algorithmes est mesurée par le **gap**, exprimé en pourcentage par rapport à l'optimum connu de TSPLIB :

$$Gap(\%) = \frac{Coût_{trouvé} - Coût_{opt}}{Coût_{opt}} \times 100$$

Instance	Opt. TSPLIB	NN	2-opt k -NN	3-opt par. (4p)	VNS hybride
berlin52	7542	19.1%	7.0%	11.6%	2.8%
ch150	6528	25.5%	8.9%	9.7%	2.6%
pcb442	50778	22.1%	7.7%	4.6%	3.9%
pr1002	259045	21.8%	7.9%	5.1%	6.7%

TABLE 3 – Évolution du gap moyen (%) par rapport à l'optimum selon l'algorithme utilisé.

Les résultats expérimentaux mettent en évidence une progression nette :

- **Baseline (NN)** : gap élevé (19–26 %), confirmant les limites d'une construction gloutonne.
- **2-opt K-NN** : gap autour de 7–9% sur l'ensemble des instances.
- **3-opt K-NN** : comportement hétérogène, légèrement moins bon que le 2-opt sur berlin52 et ch150 (≈ 10 –12%), mais meilleur sur pcb442 et pr1002 (≈ 5 %).
- **VNS hybride** : meilleur gap sur berlin52 et ch150 ($\approx 2,6$ –2,8%) ; gain plus modéré sur pcb442 (≈ 4 %) et pr1002 ($\approx 6,7$ %), où le 3-opt seul reste compétitif.

Le VNS améliore donc nettement la qualité sur les instances modestes et constitue le modèle le plus performant sur berlin52 et ch150. Sur les plus grandes instances, les gains restent limités et dépendent de la limite de nombre de kicks et de l'aléatoire.

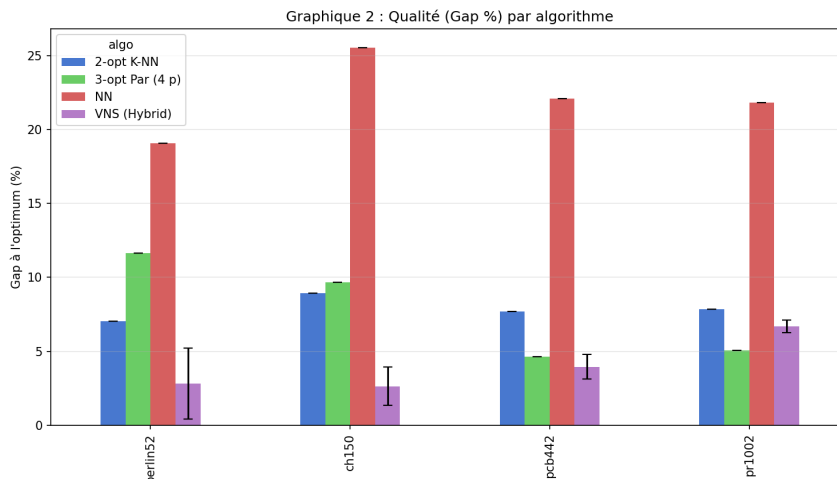


FIGURE 9 – Comparaison du gap (%) moyen par rapport à l'optimum pour chaque algorithme sur les instances testées.

L'analyse de la figure 9 confirme plusieurs tendances : le VNS (violet) obtient les gaps les plus faibles sur berlin52, ch150 et pcb442 (≈ 3 –4%), nettement devant le NN. On

distingue aussi un croisement entre 2-opt K-NN et 3-opt parallèle (4 processus) : le 2-opt reste légèrement meilleur sur les petites instances, tandis que le 3-opt devient plus compétitif sur `pcb442` et `pr1002`, où l'exploration d'un voisinage plus profond semble payer.

Sur `pr1002`, le 3-opt ($\approx 5\%$) devance le VNS ($\approx 7\%$) : la métaheuristique avec perturbations aléatoires et un nombre de kicks limité n'exploite pas toujours assez l'espace de recherche sur les très grandes instances. Les barres d'erreur (écart-type sur 5 exécutions) visibles surtout pour le VNS traduisent cette variabilité, alors que les descentes locales sont déterministes à tour initial fixé.

Le VNS reste donc l'approche la plus précise sur les instances modestes. Sur les plus grandes, le 3-opt seul peut rivaliser ou faire mieux, au prix d'un temps de calcul élevé (voir graphique temps de la figure 11).

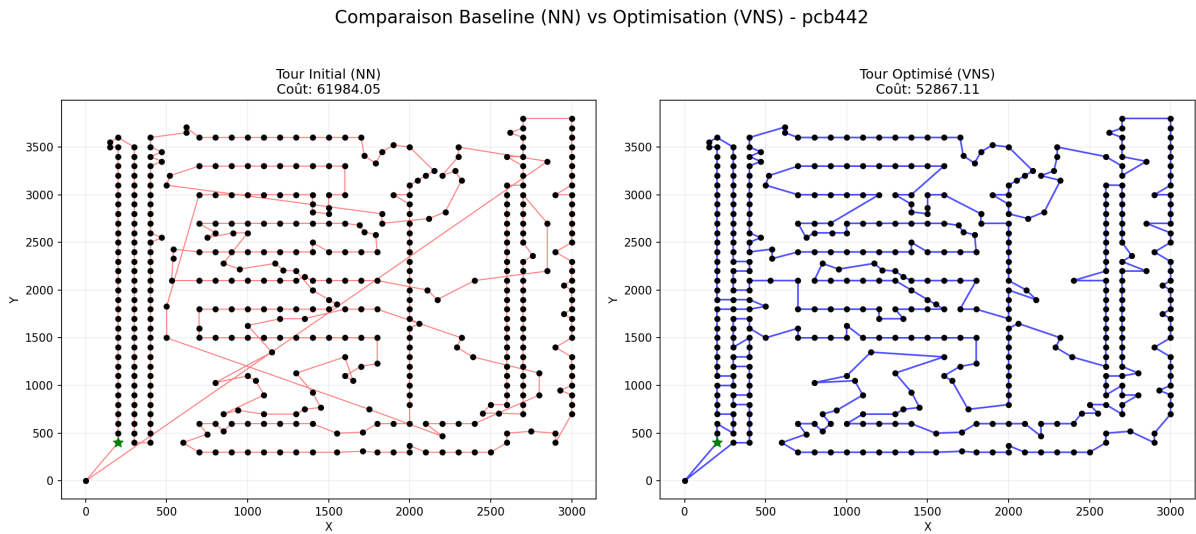


FIGURE 10 – Comparaison visuelle du tour : Baseline NN (rouge) vs Optimisation VNS (bleu) sur l'instance `pcb442`.

L'analyse visuelle de la figure 10 corrobore les résultats chiffrés. À gauche (NN), les nombreux croisements traduisent une sous-optimalité typique d'une construction gloutonne. À droite (VNS), le tour est nettement plus fluide : les "nœuds" du trajet initial sont défaits et le cycle suit mieux la géométrie des points, tout en conservant une structure globale cohérente.

7.4 Analyse du temps

Le temps d'exécution est un facteur critique, particulièrement pour les algorithmes à voisinage complexe comme le 3-opt. La figure 11 présente une comparaison des temps moyens de calcul sur une échelle logarithmique.

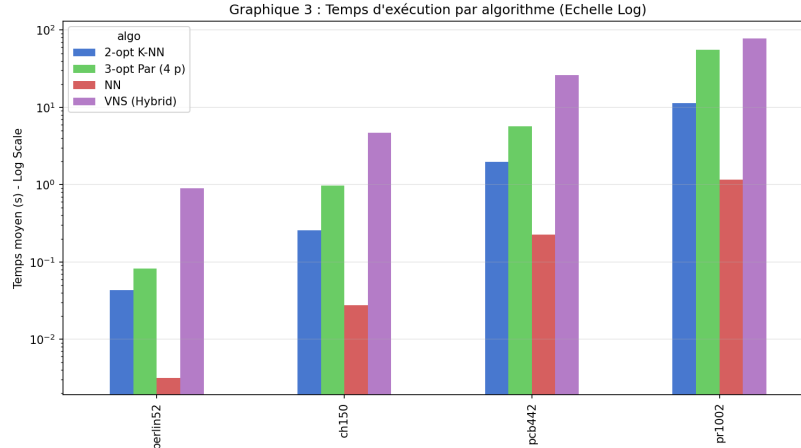


FIGURE 11 – Comparaison des temps d'exécution par algorithme (échelle logarithmique).

On observe que :

- **L'impact du k -NN** : Malgré une complexité théorique élevée, les temps restent maîtrisés même pour 1000 villes (pr1002), ce qui confirme l'intérêt du filtrage k -NN face à un voisinage 3-opt complet.
- **Coût du VNS** : Le VNS est logiquement l'algorithme le plus lent, car il orchestre de multiples descentes locales et perturbations. Cependant, ce surcoût temporel (environ 80s pour pr1002) reste tout à fait acceptable au regard de la qualité de la solution obtenue.

7.5 Analyse de Pareto et stabilité

L'évaluation finale repose sur un compromis entre deux objectifs contradictoires : minimiser l'erreur (qualité) et minimiser le temps de calcul. En analyse multi-objectif, la **frontière de Pareto**⁽¹⁹⁾ représente ici l'ensemble des solutions pour lesquelles aucune amélioration du gap n'est possible sans dégrader le temps. Sur la figure 12, chaque algorithme trace une trajectoire (*temps*, *gap*) lorsque la taille des instances augmente.

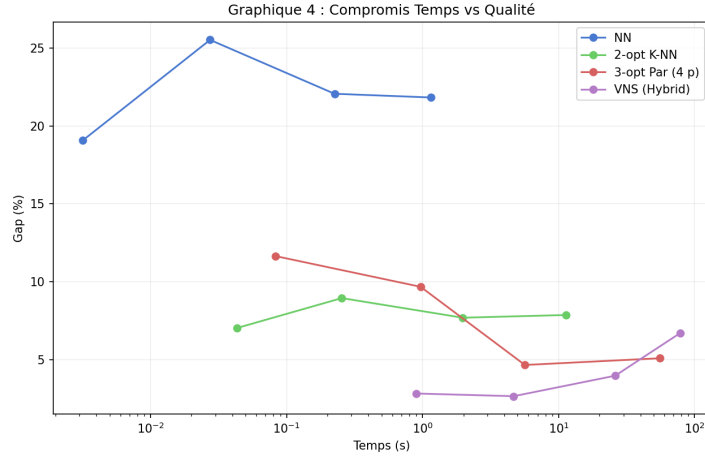


FIGURE 12 – Compromis temps vs qualité sur les quatre instances TSPLIB testées (moyennes sur 5 runs par algorithme).

Pour chaque instance, nous comparons les quatre algorithmes dans le plan (*temps*, *gap*) et retenons ceux qui sont **non dominés** (aucun autre n'est à la fois plus rapide et au moins aussi bon en qualité). Sur *berlin52*, *ch150* et *pcb442*, le VNS hybride appartient à cette frontière et réalise le gap minimal ; le NN y figure pour la rapidité maximale. Sur *pr1002*, le 3-opt parallèle (4 processus) domine le VNS (temps et gap inférieurs) : la frontière de Pareto n'est donc pas la même selon la taille de l'instance.

Le parallélisme matériel accélère donc surtout la descente 3-opt, tandis que le VNS améliore la précision sur la majorité de nos instances. Le solveur reste ainsi adaptable : 2-opt ou 3-opt pour un calcul rapide, VNS lorsque la qualité prime.

L'analyse de Pareto s'appuie sur des moyennes (gap et temps sur 5 exécutions). La figure 13 complète cette lecture sur *pr1002* en comparant la dispersion du VNS stochastique à un 2-opt *k*-NN déterministe (même tour initial, pas de perturbation aléatoire).

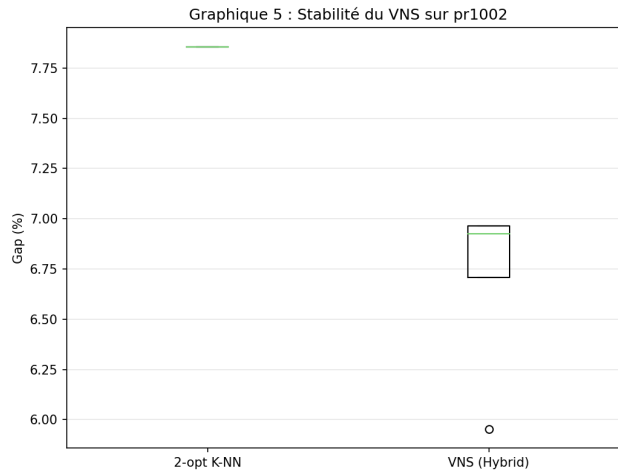


FIGURE 13 – Distribution du gap (%) sur *pr1002* pour le 2-opt *k*-NN et le VNS hybride (5 exécutions).

Le 2-opt se matérialise par un résultat unique pour un tour initial fixé. Le VNS présente une dispersion modérée, mais sa médiane (ligne verte) reste nettement inférieure à celle du 2-opt ; une exécution atypique (point isolé) atteint un gap encore plus bas. Cela confirme un gain de qualité récurrent malgré le caractère aléatoire des kicks double-bridge, sans garantir à chaque run d'améliorer le 3-opt sur les plus grandes instances.

En conclusion, l'association parallélisme (temps) et VNS (exploration) fournit un solveur polyvalent, dont le compromis temps/qualité se choisit selon la taille de l'instance et la contrainte dominante (rapidité ou précision).

8 Conclusions et perspectives

8.1 Résumé des contributions

Ce travail nous a permis de concevoir un solveur hybride pour le problème du voyageur de commerce. Les contributions principales sont les suivantes :

- **Optimisation algorithmique** : la réduction du voisinage par k -NN ramène en pratique la complexité d'une itération à $O(n \cdot k)$ pour le 2-opt et à $O(n \cdot k^2)$ pour le 3-opt (k fixé), au lieu de $O(n^2)$ et $O(n^3)$ pour l'énumération exhaustive. Pour le 2-opt, l'application par lots via un graphe de conflits permet de combiner plusieurs mouvements compatibles par itération, ce qui accélère la descente locale sans parallélisme matériel. L'exploration reste ainsi réalisable sur des instances de l'ordre du millier de villes.
- **Exploitation du parallélisme** : l'usage de `multiprocessing` et d'un tour partagé via `multiprocessing.Array` accélère la descente 3-opt (speedup d'environ 2 à 2,5 sur quatre cœurs physiques sur les grandes instances), tout en restant limité par l'overhead et la part séquentielle du code.
- **Métaheuristique hybride** : l'orchestration 2-opt par lots, 3-opt parallèle et perturbations double-bridge dans un VNS améliore nettement le gap sur la majorité des instances testées (souvent sous 4% sur `berlin52`, `ch150` et `pcb442`), au prix d'un temps de calcul plus élevé.

L'approche montre qu'une combinaison de réduction du voisinage, de parallélisme matériel et de diversification (kicks) constitue un compromis pertinent pour traiter des instances de grande taille, sans garantir l'optimalité.

8.2 Auto-évaluation et prise de recul

La réalisation de ce projet a constitué un défi technique et méthodologique particulièrement intéressant. Il nous a permis de nous immerger pour la première fois dans un véritable travail de recherche, exigeant de formuler des hypothèses crédibles et réalisables. L'exercice consistant à s'appuyer sur des ressources académiques externes, de les confronter à la réalité de l'implémentation, puis d'en extraire uniquement les éléments pertinents, a été une expérience très enrichissante.

Sur le plan technique, le parallélisme matériel en Python a représenté un enjeu majeur. Une difficulté centrale a été de contourner le GIL via `multiprocessing`, ce qui a imposé

une gestion soigneuse de la mémoire partagée (tour courant) pour limiter les surcoûts de communication. En amont, l'implémentation algorithmique elle-même a aussi demandé un soin particulier : déterminer quels mouvements 2-opt peuvent être appliqués simultanément (compatibilité des segments inversés sur le tour, via le graphe de conflits), gérer correctement les indices de rupture et les inversions de segments pour le 3-opt (découpage en quatre blocs et sept reconnections), et précalculer efficacement les distances (matrices NumPy) pour construire les listes k -NN sans pénaliser chaque itération de recherche locale.

Ce travail a aussi montré qu'une parallélisation "brute", sans réduction préalable du voisinage, apporte peu. Ce sont surtout la combinaison du filtrage k -NN, de l'application par lots du 2-opt et de l'orchestration VNS qui ont rendu l'approche tenable à l'échelle visée. Enfin, la visualisation et les benchmarks se sont révélés indispensables pour valider chaque phase du développement.

Points forts :

- Architecture modulaire et reproductible (benchmarks sur 5 runs).
- Réduction k -NN efficace.
- 2-opt par lots, accélération du 3-opt sur les grandes instances.
- VNS très compétitif sur la majorité des instances testées.

Limites :

- Le 3-opt n'est pas systématiquement supérieur au 2-opt.
- Le VNS, bien qu'efficace sur les instances modestes, peut être dominé par le 3-opt sur pr1002.
- Le parallélisme n'apporte qu'un speedup modéré et plafonne au-delà de quatre cœurs physiques.
- La métaheuristique reste partiellement aléatoire et dépend d'un nombre de kicks maximum fixé.
- L'évaluation sur quatre instances, sans comparaison à un solveur de référence, peut limiter la portée de nos conclusions.

8.3 Perspectives : GPU, métaheuristiques et apprentissage

- **Parallélisme GPU (CUDA)** : le parallélisme CPU reste limité (speedup modéré, overhead). Des travaux récents montrent que le k -opt massif sur GPU peut franchir la barrière du 3-opt et traiter des voisinages d'ordre supérieur [3]. Une extension naturelle serait d'offloader l'évaluation du voisinage 3-opt (voire 4–5-opt ciblé) sur GPU, tout en conservant le filtrage k -NN côté CPU.
- **Hybrides populationnels** : combiner notre VNS (descente + kick) avec des algorithmes génétiques ou des colonies de fourmis permettrait d'explorer plusieurs tours en parallèle et de limiter la dépendance à un seul minimum local.
- **Apprentissage pour guider la recherche** : l'utilisation de réseaux de neurones sur graphes (*Graph Neural Networks*) pour apprendre des heuristiques de construction ou de sélection de mouvements pourrait orienter le VNS vers des zones plus prometteuses de l'espace de recherche, par exemple en priorisant certains candidats k -opt ou en remplaçant le choix aléatoire des coupures du double-bridge par une des règles de décision apprises.

- **TSP dynamique / stochastique** : adapter le modèle à des distances évolutives (trafic, réseaux temps réel) imposerait une mise à jour incrémentale des voisinages et des structures partagées.

Références

- [1] M.G.A. Verhoeven, E.H.L. Aarts, P.C.J. Swinkels. *A parallel 2-opt algorithm for the traveling salesman problem*. Future Generation Computer Systems, 11(2), 175-182, 1995.
- [2] M.G.A. Verhoeven, E.H.L. Aarts. *Parallel local search*. Journal of Heuristics, 1(1), 43-65, 1995.
- [3] Wen-Bao Qiao, Jean-Charles Créput, Nan Wang, et al. *Breaking TSP local search barriers : scalable multi-GPU parallelisation of 4-opt, 5-opt, 6-opt and hybrid variable λ -opt*. Expert Systems with Applications, 296, 129110, 2026.

Annexe - Extraits de code

A : 2-opt séquentiel (first improvement)

Listing 1 – Inversion de segment et première amélioration.

```
1 def two_opt_swap(tour, i, j):
2     return tour[:i] + tour[i:j+1][::-1] + tour[j+1:]
3
4 # Boucle principale (extrait)
5 for i in range(1, num_cities - 1):
6     for j in range(i + 1, num_cities):
7         a, b = tour[i - 1], tour[i]
8         c, d = tour[j], tour[(j + 1) % num_cities]
9         current = dist(a,b) + dist(c,d)
10        new = dist(a,c) + dist(b,d)
11        if new < current - 1e-5:
12            tour = two_opt_swap(tour, i, j)
13            improvement = True
14            break
15    if improvement:
16        break
```

B : Gain 2-opt

Listing 2 – Calcul incrémental du gain.

```
1 def calcul_gain_2opt(instance, tour, i, j):
2     n = len(tour)
3     a, b = tour[i - 1], tour[i]
4     c, d = tour[j], tour[(j + 1) % n]
5     current_dist = dist(a, b) + dist(c, d)
6     new_dist = dist(a, c) + dist(b, d)
7     return current_dist - new_dist
```

C : Compatibilité des mouvements 2-opt

Listing 3 – Test de conflit entre deux segments inversés.

```
1 def mouvements_en_conflit(candidat_a, candidat_b):
2     i1, j1 = candidat_a["indices"]
3     i2, j2 = candidat_b["indices"]
4     if i1 > i2:
5         i1, j1, i2, j2 = i2, j2, i1, j1
6     return j1 + 1 >= i2
```

D : 2-opt par lots compatibles

Listing 4 – Sélection gloutonne (MIS) et application du lot.

```
1 def selectionner_lot_glouton(candidats, incompatibles):
2     lot, interdits = [], set()
3     for c in sorted(candidats, key=lambda x: x["gain"], reverse=True):
4         if c["id"] in interdits:
5             continue
6         lot.append(c)
7         interdits.add(c["id"])
8         interdits.update(incompatibles.get(c["id"], []))
9     return lot
10
11 def appliquer_lot_simultanement(tour, lot):
12     nouveau_tour = list(tour)
13     for c in sorted(lot, key=lambda x: x["indices"][0], reverse=True):
14         i, j = c["indices"]
15         nouveau_tour = two_opt_swap(nouveau_tour, i, j)
16     return nouveau_tour
```

E : Réduction du voisinage par k -NN

Listing 5 – Précalcul vectorisé des k plus proches voisins avec NumPy.

```
1 diff = coords[:, None, :] - coords[None, :, :]  
2 dist2 = (diff**2).sum(axis=-1)  
3 np.fill_diagonal(dist2, np.inf)  
4 nearest = np.argpartition(dist2, kth=k_eff - 1, axis=1)[: , :k_eff]
```

Listing 6 – Génération des candidats 2-opt restreints.

```
1 for i in range(1, n - 1):  
2     a, b = tour[i - 1], tour[i]  
3     for c in knn[a]:  
4         ajouter(i, position[c])  
5     for d in knn[b]:  
6         ajouter(i, position[d] - 1)
```

F : Reconnexions 3-opt

Listing 7 – Découpage en quatre segments et sept cas.

```
1 A = tour[:i]  
2 B = tour[i : j + 1]  
3 C = tour[j + 1 : k + 1]  
4 D = tour[k + 1 :]  
5 if cas == 1: return A + B[::-1] + C + D  
6 if cas == 2: return A + B + C[::-1] + D  
7 if cas == 3: return A + B[::-1] + C[::-1] + D  
8 if cas == 4: return A + C + B + D  
9 if cas == 5: return A + C + B[::-1] + D  
10 if cas == 6: return A + C[::-1] + B + D  
11 if cas == 7: return A + C[::-1] + B[::-1] + D
```

G : 3-opt parallèle (best improvement)

Listing 8 – Exploration k -NN d'un chunk.

```
1 for i in range(chunk_start, chunk_end):
2     for neighbor_c in _GLOBAL_KNN[tour_local[i - 1]]:
3         j = pos[neighbor_c]
4         if j < i + 2: continue
5         for neighbor_e in _GLOBAL_KNN[tour_local[i]]:
6             k = pos[neighbor_e]
7             if k < j + 2: continue
8             gain, cas = calcul_gain_3opt(instance, tour_local, i, j, k)
9             if gain > best_gain:
10                 best_gain, best_move = gain, (i, j, k, cas)
```

Listing 9 – Mémoire partagée et découpage.

```
1 shared_tour = Array('i', tour)
2 with ProcessPoolExecutor(...) as executor:
3     shared_tour[:] = tour
4     chunk_size = max(1, (n - 2) // (num_processes * 4))
5     chunks = [(i, min(i + chunk_size, n - 2))
6               for i in range(1, n - 2, chunk_size)]
7     results = executor.map(search_3opt_chunk, ...)
8     # meilleur gain parmi les workers -> best improvement
```

H : VNS et double-bridge

Listing 10 – Perturbation double-bridge.

```
1 def appliquer_4opt_double_bridge(tour, i, j, k, l):
2     A = tour[:i]
3     B = tour[i:j]
4     C = tour[j:k]
5     D = tour[k:l]
6     E = tour[l:]
7     return A + D + C + B + E
8
9 indices = sorted(random.sample(range(1, n - 1), 4))
10 tour = appliquer_4opt_double_bridge(tour, *indices)
```

Listing 11 – Boucle VNS : descente puis kick.

```
1 current_tour, cost = solve_2opt_lot_knn(instance, current_tour, k)
2 current_tour, cost = solve_3opt_knn_parallel(
3     instance, current_tour, k, num_processes=4)
4 if cost < best_cost:
5     best_tour, best_cost = current_tour, cost
6 else:
7     current_tour = perturbation_double_bridge(current_tour)
```